

"Better Prompts
Better Models
Better Future"

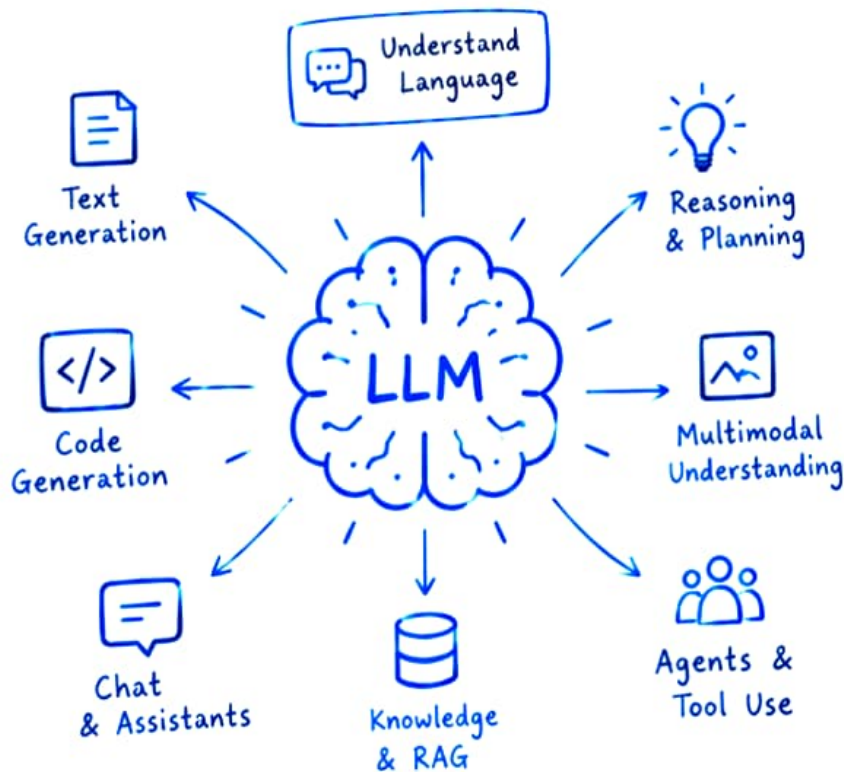
Insta ID: @techyy_bandaa

≡ **LLM** ≡

LARGE LANGUAGE MODELS

Complete Study Handbook

≡ **15 PAGES** ≡



Same Model...
Different Prompts...
Different Possibilities...



Build
Create
Explore

☆ What's Inside ?

- ① LLM Overview
- ② LLM Architectures
- ③ Pretraining LLMs
- ④ Fine-Tuning LLMs
- ⑤ LLM Training & Optimization
- ⑥ Prompt Engineering
- ⑦ LLM Inference & Decoding
- ⑧ LLM Agents & Tool Use
- ⑨ RAG & Knowledge-Augmented LLMs
- ⑩ LLM Evaluation, Safety & Alignment
- ⑪ LLM Applications & Use Cases
- ⑫ LLM Limitations & Challenges
- ⑬ Future Trends
- ⑭ Resources & Learning Path
- ⑮ Quick Revision

☆ Goal:

Understand LLMs deeply, master the key concepts, and build real-world applications with confidence.

SUBTOPIC: LLM Fundamentals

1. What is an LLM?

- LLM (Large Language Model) is a type of AI model trained on massive text data to understand, generate and manipulate human language.
- It learns patterns, grammar, facts and context from data and can produce coherent and contextually relevant responses.
- LLMs are usually based on the Transformer architecture and are trained using self-supervised learning (next token prediction).

★ **Key idea:** LLMs predict the next word/token based on the given context.

2. AI vs ML vs Deep Learning vs LLM

Aspect	AI	ML	Deep Learning	LLM
Full Form	Artificial Intelligence	Machine Learning	Deep Learning	Large Language Model
Scope	Broad	Subset of AI	Subset of ML	Subset of DL
Goal	Make machines think	Learn from data	Learn complex patterns	Understand & generate human language
Data	Varies	Structured / unstructured	Large datasets	Massive text data
Examples	Robotics, expert systems	Regression, classification	CNN, RNN, Transformers	GPT, Llama, Claude

3. Generative AI

- Generative AI refers to AI systems that can create new content (text, images, audio, code, etc.) rather than just analysing existing data.
- LLMs are a key part of Generative AI, especially for text generation.
- Uses techniques like transformers, large-scale training and prompt conditioning.

Examples:

- ChatGPT (text)
- DALL-E (images)
- Sora (videos)
- GitHub Copilot (code)

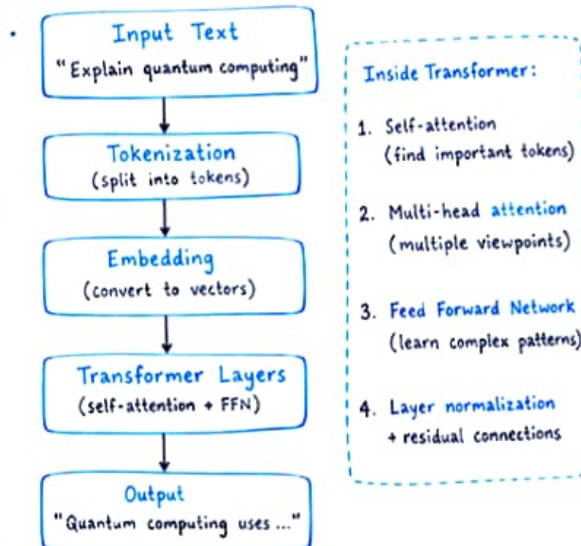
4. Foundation Models

- Foundation Models are large, general-purpose models trained on broad data and can be adapted to multiple downstream tasks.
- They have strong zero-shot and few-shot capabilities.
- LLMs like, Llama, GPT, Claude are examples of foundation models.

Key characteristics:

- Large scale
- Trained on diverse data
- Transfer learning / fine-tuning
- Multi-task capability

5. How LLMs Work



6. LLM Capabilities and Limitations

Capabilities:

- Text generation & reasoning
- Summarization
- Translation
- Code generation
- Context understanding
- Few-shot / zero-shot learning

Limitations:

- Can make factual mistakes (hallucinations)
- Limited real-world knowledge
- No true reasoning or common sense
- Fixed context window
- Biased (from training data)

7. LLM Applications

- Chatbots & virtual assistants (ChatGPT, Claude)
- Content creation (blogs, marketing, scripts)
- Coding assistance (GitHub Copilot)
- Education & tutoring (personalized learning)
- Search & knowledge retrieval (RAG)
- Healthcare (medical summarization, diagnosis support)
- Business automation (email, reports, analysis)

Real-world examples:

- Customer support bots
- Code completion in IDEs
- Document summarization
- Multilingual translation



LLMs understand context and generate meaningful responses based on patterns learned from data.

Better Models
↓
Better Applications
↓
A Smarter Future

Summary

- ★ LLMs are a powerful form of AI, built on transformers.
- ★ They are trained on massive data and can generate human-like text.
- ★ They are part of Generative AI and are foundation models.
- ★ They have strong capabilities but also important limitations.
- ★ Used in many real-world applications across industries.

SUBTOPIC: Text & Tokenization

1. Text Preprocessing

- Text preprocessing cleans and standardizes raw texts that the tokenizer and model can work effectively.
- It helps in:
 - removing noise
 - normalizing text
 - improving model performance

Common steps:

- Lowercasing (optional)
- Removing special characters (e.g. @, #, etc.)
- Handling contractions (e.g. don't → do not)
- Removing extra spaces
- Unicode normalization (e.g. é → e)

Example:

Raw: "Hello!!! This is a test... ☺"
 Cleaned: "hello this is a test"

2. Tokens and Tokenization

- Tokenization splits text into smaller units called tokens.
- Tokens can be words, subwords or characters.
- Each token is mapped to an ID from the Vocabulary.

Example:

Text: "Machine learning is amazing"

Tokens: ["Machine", "learning", "is", "amazing"]



Token types:

- Word tokens
- Subword tokens
- Character tokens

3. Word-level vs Subword Tokenization

Aspect	Word-level	Subword-level
Unit	Whole words	Subwords
Vocab size	Large (50k+ words)	Smaller (~10k-50k)
OOV handling	Poor	Better
Example	"unhappily" (1 token)	"un", "happy", "ly" (3 tokens)

Key point:

- Subword tokenization handles rare words and unknown words better than word-level.

4. BPE (Byte Pair Encoding)

- BPE starts with characters and merges the most frequent pairs iteratively.
- It creates subword units based on data frequency.
- Widely used in GPT models (e.g. GPT-2, GPT-3).

Steps:

- Start with character-level tokens.
- Find the most frequent pair.
- Merge the pair into a new token.
- Repeat until Vocab-size is reached.

Example: (simplified)

Text: "low"
 l o w → [l, o] + w → [low]

5. WordPiece

- WordPiece is similar to BPE but uses a likelihood-based merging strategy.
- It prefers merges that maximize the probability of the next token.
- Used in BERT, RoBERTa (Google).

Example:

Text: "Unhappiness"

Tokens: ["un", "happiness"]

(or ["un", "##happiness"] in some cases)

Key difference:

- BPE → frequency based
- WordPiece → likelihood based

6. SentencePiece

- A unified tokenizer that works at both word and character level.
- Uses an unsupervised learning approach (like BPE).
- Handles whitespace, punctuation and multilingual text well.
- Used in T5, mT5, LLaMA (meta).

Example:

Text: "Hello world!"

Tokens: ["Hello", "[CLS]", "world", "[SEP]"]
 (→ represents space)

7. Vocabulary

- Vocabulary is the set of unique tokens known to the model.
- Each token is mapped to a unique ID.
- Size varies based on tokenizer and model.

Typical sizes:

- Word-level: 50k - 100k
- BPE / WordPiece: 30k - 50k
- SentencePiece: 32k - 64k

Example

Token: "hello" → ID: 7592

8. Special Tokens

- Special tokens are reserved tokens used for model control and sequence handling.

Token	Purpose
[PAD]	Padding shorter sequences
[UNK]	Unknown token
[BOS]	Beginning of sequence
[EOS]	End of sequence
[MASK]	Masking (used in pretraining)

Example:

Input: [BOS] I love AI [EOS]

IDs: 101 1045 2293 280 102

9. Context Window

- The Context Window is the maximum number of tokens the model can look at in one forward pass.
- It limits how much past information the model can use.
- Larger context windows help with longer conversations and complex tasks.

Common sizes:

- GPT-3: 2K
- GPT-4: 8K / 32K
- LLaMA-2: 4K / 8K
- Claude: 100K+

Important:

- Longer context = better long-range understanding, but higher memory and compute cost.

SUBTOPIC: Embeddings & Representations

1. Word Embeddings

- Word embeddings represent words as dense vectors in a continuous space.
- Capture semantic meaning and relationships between words.
- Words with similar meanings have closer vectors.

Example:

king $\rightarrow [0.8 \ 0.2 \ -0.3 \ \dots]$ queen $\rightarrow [0.79 \ 0.21 \ -0.28 \ \dots]$ apple $\rightarrow [-0.4 \ 0.6 \ 0.1 \ \dots]$

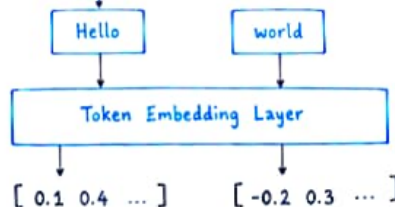
- ★ Trained using large text corpora (e.g., Word2Vec, GloVe, FastText).

2. Token Embeddings

- Token embeddings represent individual tokens (words, subwords or characters).
- Each token is mapped to a unique vector from a vocabulary.
- Used as the first step before adding positional embeddings.

Example:

Input: "Hello world"



- ★ Token embeddings are learned during training.

3. Positional Embeddings

- Provide information about the position of tokens in a sequence.
- Since attention is position-invariant, positional embeddings help the model understand word order.
- Can be learned or use fixed (sinusoidal) values.

Types:

Learned

- Trained as parameters
- More flexible

Sinusoidal

- Fixed pattern
- Allows extrapolation to longer sequences

Example (sinusoidal):

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

4. Semantic Representations

- Capture the meaning and context of text beyond just individual words.
- Include word context, sentence meaning, and even world knowledge.
- Used in tasks like text classification, QA, summarization, etc.

Example:

"bank" $\rightarrow$ financial institution"bank" $\rightarrow$ river bank

(context changes the representation)

- ★ Contextual embeddings (e.g., BERT, RoBERTa) are a type of semantic representation.

5. Dense Vectors

- Embeddings are dense vectors (most values are non-zero).
- Unlike sparse vectors (e.g., bag-of-words), dense vectors capture richer information.

Comparison:

Feature	Dense Vector	Sparse Vector
Values	Mostly non-zero	Mostly zero
Information	Richer	Limited
Memory	Higher	Lower
Example	Embeddings	Bow / TF-IDF

- ★ Dense vectors allow better semantic understanding and generalization.

6. Embedding Dimensions

- Dimension is the size of the vector.
- Higher dimensions can capture more information but increase computation and memory cost.
- Common sizes: 128, 256, 512, 768, 1024.

Trade-off:

Lower Dimensions	Higher Dimensions
✓ Faster	✓ More expressive
✓ Less memory	✓ Better performance
✗ Less expressive	✗ Higher cost

- ★ Typical range: 128 - 1024 (depends on model and task).

7. Similarity Measures

- Used to find how close two embeddings are in the vector space.
- Common measures:
 - Cosine Similarity (most common)
 - Euclidean Distance
 - Dot Product
 - Manhattan Distance

Example (Cosine Similarity):

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

- ★ Higher similarity = more related meaning.

8. Embedding Models

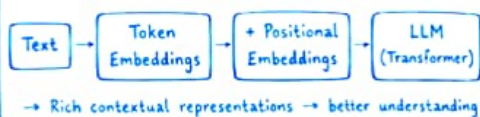
- Pre-trained models that generate embeddings for text (words, sentences, documents).
- Capture semantic and contextual information.
- Popular models:
 - Word2Vec (word-level)
 - GloVe (word-level)
 - BERT (contextual)
 - RoBERTa (contextual)
 - Sentence-BERT (sentence-level)
 - OpenAI Embeddings (text/semantic search)

- ★ These models are used in RAG, search, clustering, recommendation and more.

Quick Revision

- Word embeddings $\rightarrow$ words as vectors
- Token embeddings $\rightarrow$ token level
- Positional embeddings $\rightarrow$ word order
- Semantic representations $\rightarrow$ context + meaning
- Dense vectors $\rightarrow$ mostly non-zero
- Embedding dimensions $\rightarrow$ size of vector
- Similarity measures $\rightarrow$ cosine, dot, euclidean
- Embedding models $\rightarrow$ Word2Vec, BERT, etc.

Overall Flow



SUBTOPIC: Transformer Architecture

1. Transformer Overview

- Transformer is a neural network architecture introduced in "Attention is All You Need" (2017).
- It replaces recurrence and convolutions with self-attention mechanisms.
- Highly parallelizable, scalable and achieves state-of-the-art performance in NLP tasks.

Key idea:

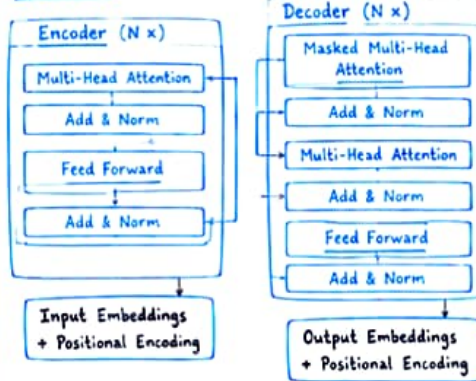
Use self-attention to capture long-range dependencies in data.



2. Encoder and Decoder

- Encoder:** Converts input sequence into contextual representations.
- Decoder:** Generates output sequence (e.g., translation, summarization).

Structure:



3. Self-Attention

- Self-attention lets each token look at all tokens in the sequence.
- Computes attention scores to weigh the importance of other tokens.

Formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where:

Q = Query matrix

K = Key matrix

V = Value matrix

d_k = dimension of keys/queries

Example:

"The cat sat on the mat"

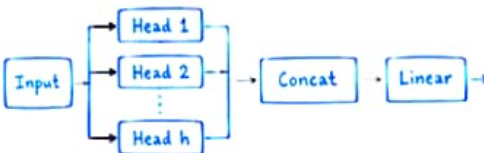
→ each word attends to all words.

4. Multi-Head Attention

- Uses multiple attention heads to capture different relationships (syntactic, semantic, etc.).
- Each head learns different patterns.
- Outputs are concatenated and projected.

Steps:

- Linear projections for Q, K, V (for each head)
- Apply self-attention in parallel
- Concatenate outputs
- Final linear layer



5. Feed-forward Networks

- Two linear layers with a non-linear activation (ReLU/GELU).
- Applied to each position independently and identically.
- Expands and then projects back to original size.

Formula:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (\text{ReLU})$$

or

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (\text{GELU})$$

Example (typical sizes):

$$d_{\text{model}} = 512 \rightarrow d_{\text{ff}} = 2048$$



6. Residual Connections

- Adds the input of a sublayer to its output.
- Helps with gradient flow and training deep networks.
- Used around both attention and feed-forward sublayers.

Formula:

$$y = \text{LayerNorm}(x + \text{Sublayer}(x))$$



★ Residual connection = better gradient flow + faster convergence.

7. Layer Normalization

- Normalizes inputs across features (not batch).
- Reduces internal covariate shift.
- Applied after residual connections.

Formula:

$$\text{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

Where:

μ = mean, σ^2 = variance

γ, β = learnable parameters

Note:

- Used instead of BatchNorm in Transformers.

8. Positional Encoding

- Self-attention is position agnostic.
- Positional encoding provides information about token positions.
- Can be learned or fixed (sinusoidal).

Sinusoidal (common):

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

$$\text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d_{\text{model}}}}\right)$$

Example:

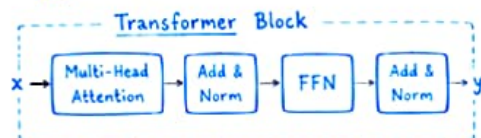
For $d_{\text{model}} = 512$, position 0, 1, 2, ...

→ adds unique vectors to each position.

9. Transformer Block

- A single layer of the transformer (encoder or decoder).
- Consists of:

- Multi-Head Attention
- Add & Norm
- Feed-forward Network
- Add & Norm



Key Takeaways:

- Attention enables global context.
- Residual + LayerNorm = stable training.
- Positional encoding adds order information.
- Stack multiple blocks for deeper models.

SUBTOPIC: Attention Mechanism

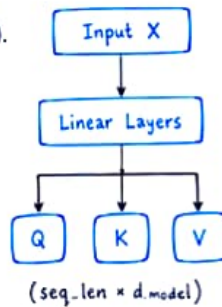
1. Query, Key, Value

- In attention mechanism, each token is converted into three vectors: **Query (Q)**, **Key (K)** and **Value (V)**.
- Q** is used to ask a question, **K** is used to match with other tokens and **V** contains the information to be aggregated.
- These are obtained using linear transformations of the input embeddings.

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$

Where:

- X** : input embeddings (seq_len × d_model).
- W_Q, W_K, W_V** : learnable weight matrices



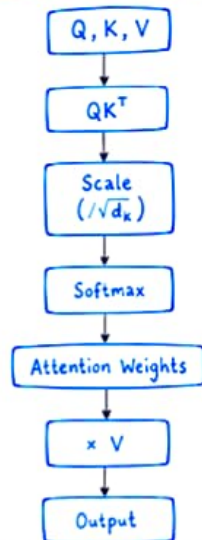
2. Scaled Dot-Product Attention

- Computes attention scores using dot product between queries and keys, then scales the result and applies softmax to get weights.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \cdot V$$

Where:

- Q** : Query matrix (seq_len × d_k)
- K** : Key matrix (seq_len × d_k)
- V** : Value matrix (seq_len × d_v)
- d_k** : dimension of keys/queries



3. Self-Attention

- In self-attention, the model attends to all tokens in the same sequence (including itself).
- Helps capture context and long-range dependencies.

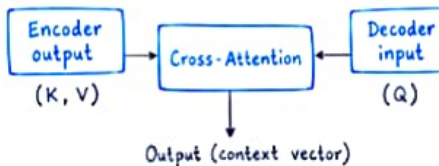


Example:



4. Cross-Attention

- Used in encoder-decoder models (e.g., translation).
- Queries come from the decoder, while keys and values come from the encoder output.



Example:

- Helps the decoder focus on relevant parts of the input sequence.

5. Multi-Head Attention

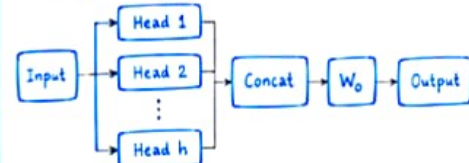
- Uses multiple attention heads in parallel to learn different representations and relations.
- Each head has its own **Q, K, V** projections.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

Where:

- h** : number of heads
- W_O** : output projection matrix



6. Causal Attention

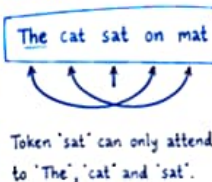
- Used in decoder-only models (e.g., GPT).
- Prevents attending to future tokens.
- Implemented using a **causal mask** (upper triangular).

Attention Mask

	1	2	3	4	5
1	0	-∞	-∞	-∞	-∞
2	0	0	-∞	-∞	-∞
3	0	0	0	-∞	-∞
4	0	0	0	0	-∞
5	0	0	0	0	0

(1 = attend, -∞ = masked)

Example:



Token 'sat' can only attend to 'The', 'cat' and 'sat'.

7. Attention Masks

- Masks control which tokens can be attended to.
- Types**: padding mask, causal mask, or combined.
- Used to ignore and prevent future attention.

Types:

- Padding mask → ignore padded positions
- Causal mask → block future tokens
- Combined mask → both padding + causal

$$\text{Masked Attention} = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right)$$

Where:

- M** : mask matrix (0 or -∞)

8. Computational Complexity

- Attention is efficient but still expensive, especially for long sequences.
- For a sequence of length **n** and model dimension **d**:

$$\text{Time Complexity: } O(n^2 \cdot d)$$

$$\text{Space Complexity: } O(n^2)$$

Breakdown:

- QK^T** : $O(n^2 \cdot d)$
- Softmax** : $O(n^2)$
- Multiply with V** : $O(n^2 \cdot d)$

Note:

- For very long sequences, smarter, efficient variants (e.g. sparse attention, linear attention) are used to reduce complexity.

★ Attention allows the Model to focus on relevant information, improving context understanding and performance. ★

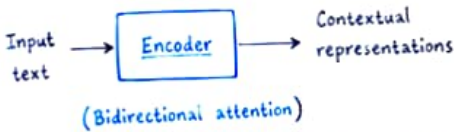
SUBTOPIC: LLM Architectures

1. Encoder-only Models

- Process the entire input sequence at once.
- Good at understanding context (bidirectional).
- Usually used for classification, feature extraction, and embedding generation.

Examples:

- BERT, RoBERTa, ALBERT, DeBERTa.



Use cases:

- Text classification
- Named entity recognition
- Semantic search
- Embeddings

2. Decoder-only Models

- Generate text token by token (causal / left-to-right).
- Only attend to previous tokens (masked attention).
- More efficient for generation tasks.

Examples:

- GPT-1, GPT-2, GPT-3, GPT-4, LLaMA, Mistral.



Use cases:

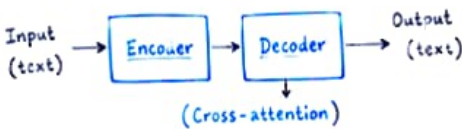
- Text generation
- Chatbots
- Code generation
- Summarization

3. Encoder-Decoder Models

- Combines encoder (understanding) + decoder (generation).
- Encoder reads the input, decoder generates output.
- Uses cross-attention to connect both.

Examples:

- T5, BART, M2S, Pegasus.



Use cases:

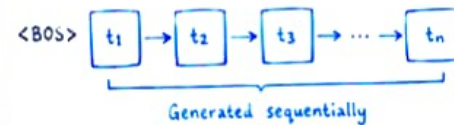
- Translation
- Summarization
- Question answering
- Text generation with context

4. Autoregressive Models

- Generate output sequentially, one token at a time.
- Each new token depends on previous tokens.
- Trained using next-token prediction.

Examples:

- GPT, LLaMA, Mistral, Claude.



Key point:

- Uses causal attention / mask.
- Ensures left-to-right generation.

5. Causal Language Models

- A type of autoregressive model with a strict mask.
- Prevents attending to future tokens.
- Ensures generation to continue during training and inference.

Examples:

- GPT, LLaMA, Mistral.

	t ₁	t ₂	t ₃	t ₄
t ₁	1	0	0	0
t ₂	1	1	0	0
t ₃	1	1	1	0
t ₄	1	1	1	1

Masking:

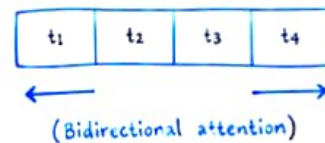
- Future tokens are masked (set to -inf).
- Uses triangular mask.

6. Bidirectional Models

- Can attend to both left and right context.
- Better at understanding the overall meaning.
- Used in encoder-style models and some hybrid models.

Examples:

- BERT, RoBERTa, DeBERTa.



Use cases:

- Masked language modeling
- Text classification
- Feature extraction
- Embeddings

7. Mixture-of-Experts (MoE)

- Uses multiple expert networks, with a gating mechanism.
- Activates only a few experts per token.
- Improves efficiency and scalability.

Architecture:



Benefits:

- Higher capacity
- Better scalability
- Faster training (sparse activation)

Examples: Mixtral, GLaM, Switch Transformer.

8. Sparse Architectures

- Use sparse connections instead of dense ones.
- Activates only a part of the model for each input.
- Reduces memory and compute cost.

Types:

- Sparse attention
- Sparse MoE
- Sparse feed-forward

Advantages:

- Lower memory usage
- Faster computation
- Scales to larger models

Examples: Longformer, BigBird, Sparse Transformer.

★ Different architectures trade off between context understanding, efficiency and generation quality. ★

SUBTOPIC: Pretraining LLMs

1. Language Modeling

- Language modeling is the task of predicting the next token in a sequence of text.
- It learns the probability distribution of the next token given the previous tokens.
- LLMs are trained using large-scale language modeling objectives massive text data.

Formula:

$$P(x) = \prod_{i=1}^n P(x_i | x_{<i})$$

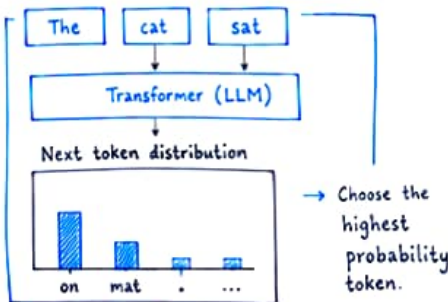
Where:

- x_i = next token
- $x_{<i}$ = previous tokens

Example: "The cat sat" → predict "on"

2. Next-token Prediction

- The Model takes a sequence of tokens and predicts the next token using a softmax layer.
- Trained with cross-entropy loss.



Loss (cross-entropy):

$$L = - \sum_{i=1}^V y_i \log(\hat{y}_i)$$

3. Pretraining Datasets

- Large, diverse text corpora are used (web pages, books, code, academic papers, etc.).
- The data is usually in raw form, and needs significant preprocessing.

Common sources:

- Web (Common Crawl, Wikipedia)
- Books (Project Gutenberg)
- Code (GitHub, Stack Overflow)
- Academic papers

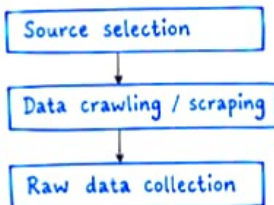
Key point:

- Quality, diversity and scale of data directly affect model performance.

4. Data Collection

- Gather text from multiple sources (web, books, code, etc.).
- Use web crawlers, APIs, or licensed datasets.
- Filter out low-quality or irrelevant content.

Steps:



Example: Common Crawl, Wikipedia API, GitHub, etc.

5. Data Cleaning

- Remove noise, html tags, special characters, and unwanted content.
- Normalize text (lowercase, remove extra spaces).
- Handle encoding issues and broken text.

Common techniques:

- Text normalization
- Language filtering
- Remove boilerplate (e.g., headers, footers)

Example:

"<p>Hello!! World!!!</p>"
→ "hello world"

6. Data Deduplication

- Removes duplicate or near-duplicate content to avoid and reduce training cost.
- Uses hashing, similarity matching or MinHash techniques.

Methods:

- Exact match (hashing)
- Near-duplicate (SimHash, MinHash)
- Document-level deduplication

Example:

"The cat is on the mat."
The cat is on the mat!"
→ Keep one, remove duplicate.

7. Training Objectives

- Goal: learn better representations and generalize well.
- Common objectives:
 - Causal LM (next-token prediction)
 - Masked LM (e.g., BERT-style)
 - Span / sequence objectives (e.g., T5)
 - Contrastive loss (e.g., CLIP)

Note:

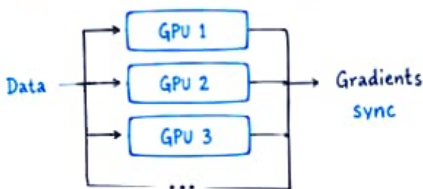
- Different models use different objectives based on their architecture and goals.

8. Distributed Training

- Splits training across multiple GPUs/servers.
- Increases batch size and training speed.
- Uses frameworks like PyTorch DDP, DeepSpeed, Megatron-LM.

Types:

- Data parallelism (split data)
- Model parallelism (split model)
- Pipeline parallelism (split layers)



9. Compute Requirements

- Pretraining requires massive compute and memory.
- Typically uses large GPU clusters or TPUs.
- Training time can range from days to weeks depends on model size.

Key factors:

- Model size (parameters)
- Context length
- Batch size
- Precision (FP32 / FP16 / BF16)
- Hardware (GPUs / TPUs)

Typical scale (example):

- 100M - 1B parameters → smaller models
- 10B - 100B parameters → large models
- 100B+ parameters → frontier models

★ High-quality data + effective preprocessing + good training objectives + distributed training + adequate compute
= Better pretraining and stronger LLMs ★

SUBTOPIC: LLM Training & Optimization

1. Gradient Descent

- Gradient descent updates parameters in the direction of negative gradient of the loss function.
- Used to minimize the training loss.

Update rule:

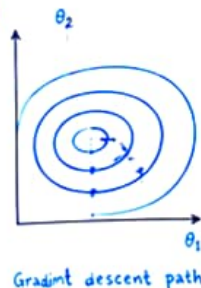
$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Where:

- θ = model parameters
- η = learning rate
- L = loss function

Example:

$$\theta_{t+1} = \theta_t - 0.001 \nabla L(\theta_t)$$

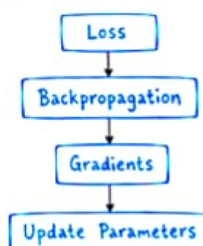


2. Backpropagation

- Computes gradients of the loss w.r.t. model parameters using chain rule.
- Propagates error from output to input layer.
- Enables efficient training of deep networks.

Steps:

- Forward pass (compute output & loss)
- Backward pass (compute gradients)
- Update parameters (using optimizer)



Uses chain rule:

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial \theta}$$

3. Optimizers

- Optimizers control how parameters are updated using gradients.
- Help with faster convergence and better performance.

Common optimizers:

- SGD (with/without momentum)
- Adam
- AdamW
- RMSprop

AdamW is usually preferred for LLMs due to better generalization.

4. Adam / AdamW

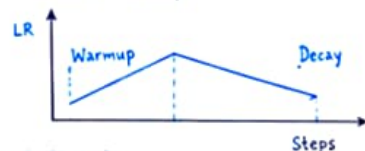
- Adam = adaptive learning rate + momentum.
- AdamW = Adam with decoupled weight decay.
- Helps in faster and stable convergence.
- Better for large-scale training.

Key difference: Weight decay is applied separately in AdamW (not mixed with gradient).

5. Learningrate Scheduling

- Adjusts learning rate during training.
- Helps with better convergence and stability.
- Common strategies:

- Step decay
- Cosine annealing
- Linear decay
- Warmup + decay



Example (Cosine):

$$\eta_t = \eta_{\min} + \frac{1}{2} (\eta_{\max} - \eta_{\min}) \left(1 + \cos \left(\frac{\pi t}{T} \right) \right)$$

Where:

- t = current step
- T = total steps

6. Batch Size

- Number of samples processed before updating model parameters.
- Smaller batch $\rightarrow$ More noisy gradients.
- Larger batch $\rightarrow$ More stable gradients.
- Trade-off between speed, memory and generalization.

Batch Size	Pros	Cons
Small	- Better generalization - More noise (escape local minima)	- Slower training - Higher variance
Medium	- Good balance - Stable training	Moderate memory usage
Large	- Faster training - Better hardware utilization	- Higher memory requirement - May reduce generalization

7. Training Epochs

- One epoch = one full pass over the dataset.
- More epochs $\rightarrow$ better learning (until convergence).
- Too many epochs $\rightarrow$ overfitting.



Typical range:

- 1 - 10 for small datasets
- 10 - 100+ for large-scale LLM training

8. Mixed Precision

- Uses FP16 / BF16 instead of FP32.
- Reduces memory usage and speeds up training.
- No significant loss in model quality (with loss scaling).

Benefits:

- Faster training
- Lower memory usage
- Better GPU utilization

Tools:

- torch.cuda.amp
- GradScaler

Note: BF16 is preferred for stability in LLM training.

9. Distributed Training

- Trains model across multiple GPUs / nodes.
- Reduces training time and handles larger models.
- Uses data parallelism / model parallelism / pipeline parallelism.

Types:

- Data Parallelism (DDP)
- Model Parallelism (split model)
- Pipeline Parallelism (split layers)



Key tools: PyTorch DDP, FSDP, DeepSpeed

10. GPU / TPU Training

- GPUs:** widely used, high memory bandwidth, good for mixed precision.
- TPUs:** optimized for matrix operations, high throughput, used in large-scale training.

Feature	GPU	TPU
Memory	High	High
Speed	High	Very high
Precision	FP16/BF16	BF16
Availability	Cloud / Local	Cloud (mainly)

Use cases:

- GPUs: general-purpose, research, smaller scale.
- TPUs: large-scale LLM training (e.g., GPT, PaLM).

★ Efficient training requires the right combination of data, model, optimizer and hardware. ★

SUBTOPIC: Fine-Tuning LLMs

1. Pretraining vs Fine-Tuning

- Pretraining:** Learns general knowledge and language patterns from large, diverse text data.
- Fine-tuning:** Adapts the pretrained model to a specific task, domain or instruction style using smaller, task-specific data.

Key differences:

Aspect	Pretraining	Fine-tuning
Data	Large, diverse corpora	Task-specific data
Objective	Learn general patterns	Improve task performance
Training cost	High	Lower
Model state	Base model	Task-specific model

Example: Pretrain on CommonCrawl → Fine-tune on instruction dataset (e.g., Alpaca, ShareGPT).

2. Supervised Fine-Tuning (SFT)

- Trains the model on labeled input-output pairs.
- Helps the model follow instructions and produce desired responses.

Training format:

Input: < instruction / prompt >

Output: < desired response >

Loss:

$$L_{\text{SFT}} = -\frac{1}{N} \sum_{i=1}^N \log p(y_i | x_i)$$

Example: (instruction → response)

Input: What is photosynthesis?

Output: Photosynthesis is the process...

3. Instruction Tuning

- Special form of fine-tuning where the model learns to follow natural language instructions.
- Uses instruction response pairs, often with human preferences.

Example formats:

- "Summarize this text..."
- "Answer the question..."
- "Write a code for..."

Benefits:

- Improves alignment with human intent
- Makes the model more versatile

Common datasets: FLAN, TO Dolly, ShareGPT, OpenAssistant.

4. Parameter-Efficient Fine-Tuning (PEFT)

- Updates only a small subset of parameters.
- Reduces memory, compute and training time.
- Keeps most of the pretrained weights frozen.

Common methods:

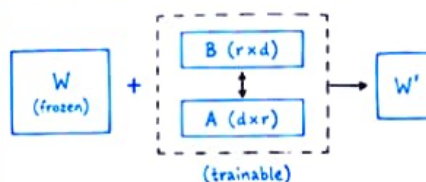
- LoRA
- QLoRA
- Adapters
- Prefix / Prompt tuning

Useful when:

- GPU memory is limited
- Training data is small
- Fast adaptation is needed

5. LoRA (Low-Rank Adaptation)

- Adds low-rank matrices to the attention layers.
- Freezes original weights, trains only LoRA params.
- Large reduction in trainable parameters.



$$W' = W + \Delta W \quad \text{where } \Delta W = BA$$

Benefits: efficient, minimal memory, good performance.

Common use: LLMs, vision models.

6. QLoRA

- Combines LoRA with 4-bit quantization.
- Quantizes the base model (e.g., NF4).
- Trains LoRA adapters in 16-bit precision.

Benefits:

- Even lower memory usage
- Enables larger models on limited GPUs
- Comparable performance to full fine-tuning



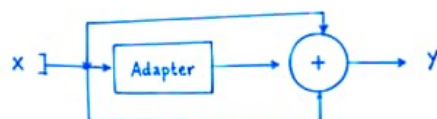
Use case: Fine-tuning large models (e.g., T3, 13B) on single GPU.

7. Adapters

- Learnable modules inserted into each layer.
- Original model weights are frozen.
- Can be placed in different parts (e.g., after attention or FFN).

Types:

- Houlsby Adapter (serial)
- Pfeiffer Adapter (parallel)
- Compacter (parameter efficient)



Benefits: Modular, flexible, good for multi-tasking.

Use case: multi-task learning, domain adaptation.

8. Full Fine-Tuning

- Updates all model parameters.
- Highest performance potential.
- Requires more memory and compute.

Pros:

- Best task specific performance
- Learns deeper task specific patterns

Cons:

- Expensive (memory + compute)
- Higher risk of overfitting



Use case: When you have enough data and compute resources.

9. Fine-Tuning Datasets

- High-quality, task relevant data is crucial.
- Includes instruction datasets, domain-specific data, and human-annotated data.

Common sources:

- Open source (Alpaca, ShareGPT, FLAN)
- Synthetic data (self instruct, distillation)
- Custom / proprietary data

Dataset Type	Examples
Instruction	Alpaca, Dolly, ShareGPT
Domain-specific	Medical, Legal, code
Preference (RLHF)	Anthropic HH, UltraFeedback
Synthetic	Self-Instruct, Distilabel

- Good datasets improve alignment, reduce hallucinations and boost performance.

★ Right data + right method + efficient training = Better adaptation and performance ★

SUBTOPIC: Prompt Engineering

1. Zero-shot prompting

- The model is given a task without any examples.
- Relies on the model's pretraining and general knowledge.
- Useful for simple and well-defined tasks.

Example:

Prompt: "Translate the following sentence to French: Hello, how are you?"

Output: "Bonjour, comment ça va?"

2. Few-shot prompting

- Provides a few input-output examples in the prompt.
- Helps the model understand the pattern or desired format.
- Works well for tasks with specific styles or formats.

Example:

Prompt: "Example 1: $2 + 3 = 5$

Example 2: $10 - 4 = 6$

Example 3: $7 + 8 = ?$ "

Output: "15"

3. Role prompting

- Assigns a specific role or persona to the model.
- Helps generate more relevant, focused and domain-specific responses.
- Useful for creative, professional or expert-like outputs.

Example:

Prompt: "You are a nutrition expert.

Suggest a healthy breakfast for 20 year old student."

Output: "Here's a nutritious breakfast: oats, fruits, nuts and milk...."

4. System prompts

- Sets the overall behavior, rules or instructions for the model.
- Used in chat-based applications to guide the assistant's responses.
- Works together with user prompts.

System prompt: "You are a helpful, concise and polite assistant."

User prompt: "Explain photosynthesis in simple terms."

5. Chain-of-thought (CoT)

- Encourages the model to reason step-by-step.
- Improves accuracy on complex problems (math, logic, multi-step tasks).
- Can be triggered with phrases like "Let's think step by step".

Prompt: "Solve this step by step:
If a train travels 60 km in 1 hour,
how far will it travel in 3.5 hours?"

Output: "Step 1: $60 \times 3.5 = 210$ km
So, the train will travel 210 km."

6. Structured prompting

- Requests output in a specific format (JSON, table, list, etc.).
- Helps with data extraction and downstream processing.
- Reduces ambiguity in responses.

Prompt: "List 3 benefits of exercise in JSON format."

Output: { "benefits": ["Improves health", "Boosts mood", "Increases energy"] }

7. Prompt templates

- Predefined reusable prompts with placeholders.
- Saves time and ensures consistency.
- Useful for applications and production systems.

Template: "Summarize the following text: {text} in {length} words."

(Replace {text} and {length} with values)

8. Prompt optimization

- Iteratively improve prompts for better results.
- Techniques: rephrasing, adding examples, adjusting temperature, changing format.
- Can use human feedback or automated evaluation.

Optimization process:



9. Common prompting techniques

Technique	Use case
Zero-shot	General tasks
Few-shot	Pattern learning
Role prompting	Domain-specific outputs
System prompt	Control behavior
CoT	Complex reasoning
Structured	Structured output
Templates	Consistency & scaling
Optimization	Better performance

★ Key tips

- Be clear and specific in your instructions.
- Use examples when needed.
- Try different phrasings and formats.
- Use system prompts for chat models.
- Evaluate and iterate for better results.

★ Common mistakes

- Vague or ambiguous prompts.
- Too much information in one prompt.
- Wrong output format expectations.
- Not using examples for complex tasks.
- Ignoring model behavior (temperature, etc.).

★ Quick revision

- ✓ Zero-shot = no examples
- ✓ Few-shot = few examples
- ✓ Role = persona/role
- ✓ System = overall behavior
- ✓ CoT = step-by-step reasoning
- ✓ Structured = specific format
- ✓ Templates = reusable prompts
- ✓ Optimization = iterative improvement

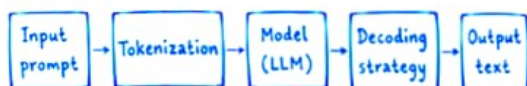
★ Good prompting is skill. The clearer your prompt, the better the response! ★

SUBTOPIC: LLM Inference & Decoding

1. LLM Inference Pipeline

- Inference is the process of generating a response using a pre-trained (and optionally fine-tuned) model.

Pipeline:



Steps:

- Convert text to tokens.
- Pass tokens through the model (forward pass).
- Generate next token(s) using a decoding strategy.
- Detokenize to get final output.

Example:

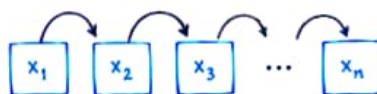
Input: "Explain photosynthesis."

Output: "Photosynthesis is the process by which..."

2. Autoregressive Generation

- Generates text one token at a time.
- Each new token is predicted based on previous tokens.
- Formulation:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | x_{<i})$$



Next token depends on previous tokens.

Key point:

Used in most LLMs (e.g., GPT, LLaMA).

3. Greedy Decoding

- At each step, choose the token with the highest probability.
- Simple and fast, but can't be repetitive or less diverse.
- Algorithm:
 - Compute probabilities for next tokens.
 - Pick token with maximum probability.
 - Repeat until end token or Max length.

Example:

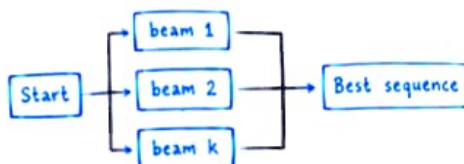
Probabilities: {"cat": 0.4, "dog": 0.3, "bird": 0.2}
→ Pick "cat" (highest).

Pros / Cons:

- + Fast, simple, low compute
- May produce repetitive / generic output

4. Beam Search

- Keeps multiple candidate sequences (beams) at each step.
- Explores more possibilities than greedy decoding. Uses a beam width (k) to control search size.
- Algorithm:
 - Keep top-k tokens at each step.
 - Expand each beam with next token.
 - Continue for fixed steps or until end token.



Key point:

Better quality than greedy, but higher cost.

5. Temperature

- Controls randomness in token selection.
- Higher temperature → More random, diverse output.
- Lower temperature → More focused, deterministic output.

$$P'(x_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

$T < 1$ → sharper distribution (focused)

$T = 1$ → original distribution

$T > 1$ → flatter distribution (more random)

Example:

$T = 0.7$ → More deterministic

$T = 1.5$ → More creative

6. Top-K Sampling

- Consider only the top-k tokens with highest probability.
- Sample from this subset (using Softmax).
- Prevents low-probability tokens from being chosen.



Example:

$K = 50$ → Sample from top 50 tokens.

$K = 5$ → More focused, less diverse.

Use case:

- Good balance of quality and diversity.

7. Top-P (Nucleus) Sampling

- Keep the smallest set of tokens whose cumulative probability $\geq p$.
- Sample from this dynamic set.
- Adapts to different contexts.
- Algorithm:
 - Sort tokens by probability.
 - Find smallest set with cumulative $\geq p$.
 - Sample from this set.

Example:

$p = 0.9$ → Loss tokens until 90% probability.

$p = 0.7$ → More diverse output.

8. Repetition Penalty

- Discourages repeating the same tokens.
- Increases the probability denominator for already generated tokens.
- Common values: 1.1 - 1.5

Formula:

$$\tilde{P}(t_i) = \begin{cases} P(t_i) / r & \text{if } t_i \text{ was generated} \\ P(t_i) & \text{otherwise} \end{cases}$$

$r > 1$ (penalty)

Effect:

- Reduces repetitive / looping text.
- Improves diversity and coherence.

9. KV Cache

- Stores past Key and Value tensors for each layer.
- Avoids recomputing attention for previous tokens.
- Reduces latency and memory compute.



Benefits:

- Faster generation (lower latency).
- Less compute and memory usage.
- Common in production LLMs.

10. Latency and Throughput

Latency

- Time to generate one output (e.g., per token).
- Affected by model size, decoding strategy, hardware, and sequence length.
- Lower latency = better real-time use.

Throughput

- Number of tokens generated per second.
- Depends on batch size, hardware, and optimization.
- Higher throughput = better for large-scale inference.

Typical values:

- Latency: 10 - 500 ms/token
- Throughput: 50 - 10,000+ tokens/sec (depending on setup)

SUBTOPIC: RAG & Knowledge-Augmented LLMs

1. Retrieval-Augmented Generation

- Combines external knowledge retrieval with LLM generation.
- Helps reduce hallucinations and provides up-to-date information.
- Works by retrieving relevant context and passing it to the LLM as additional input.

RAG Pipeline:



Example:

Query: "What is the capital of France?"
 → Retrieve from knowledge base → Add context
 → LLM generates: "The capital of France is Paris."

2. Document Ingestion

- Load data from various sources (PDF, web, docs, etc.).
- Convert to a machine-readable format.
- Extract text, metadata and structure.

Typical sources:

- PDFs / Word / Markdown
- Websites / APIs
- Databases / Cloud storage

Pipeline:



Key point:

- Handle different file formats and preserve important metadata.

3. Chunking

- Splits large documents into smaller pieces (chunks).
- Improves retrieval accuracy and relevance.
- Uses different strategies (fixed size, semantic, recursive, etc.).

Common strategies:

- Fixed size (e.g., 500 tokens)
- Overlapping chunks (e.g., 200 overlap)
- Semantic / topic-based
- Recursive chunking

Example:

"The cat is a pet. It is small."
 → "The cat is a pet." "It is small."

4. Embeddings

- Convert text into dense vector representations.
- Capture semantic meaning and context.
- Used for both documents and queries.

Types:

- OpenAI embeddings
- Sentence Transformers
- BGE / E5 / etc.

Formula (conceptual):

text → encoder → vector (d-dim)

Example:

"machine learning" → [0.12, -0.03, 0.45, ...]

5. Vector Databases

- Store embeddings for efficient similarity search.
- Support large-scale and fast retrieval.
- Examples: Pinecone, Weaviate, FAISS, Milvus.

Database	Type	Key Feature
Pinecone	Managed	Easy to use, scales well
Weaviate	Open source	Hybrid search (vector + keyword)
FAISS	Open source	High performance, large scale
Milvus	Open source	Enterprise grade

Key point:

- Choose based on scale, features and budget.

6. Similarity Search

- Finds most relevant documents using vector similarity.
- Common metrics: cosine, dot_product, L2.

Formulas:

Cosine similarity:

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\| \|b\|} \quad \left\{ \begin{array}{l} \text{Higher value =} \\ \text{more similar} \end{array} \right.$$

L2 distance:

$$d(a, b) = \|a - b\|_2$$

Use case:

- Find top-k most relevant chunks for a query.

7. Retrieval

- Fetches top-k relevant documents/chunks from the vector database.
- Uses similarity for each to find matches.
- Can include filters (metadata, time, etc.).

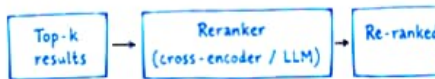


Key point:

- Balance between recall (more results) and precision (better quality).

8. Reranking

- Reorders retrieved results using a better model.
- Improves relevance of final context.
- Uses cross-encoder or LLM-based rerankers.



Example:

- Initial: [doc1, doc2, doc3, ...]
- After rerank: [doc2, doc1, doc3, ...]

Benefit:

- More accurate and relevant context.

9. Context Injection

- Adds retrieved context to the LLM input.
- Can use simple concatenation or advanced methods (e.g., templates).
- Keeps the model grounded in external knowledge.

Prompt template:

"Use the following context to answer the query."

Context: {retrieved_chunks}

Query: {user_query}

Key point:

- Keep context concise and relevant.

10. RAG Evaluation

- Measures quality, relevance and factuality.
- Uses automatic and human evaluation.
- Key metrics: faithfulness, answer relevance, context relevance, latency.

Metric	Description
Faithfulness	Does it stick to retrieved info?
Answer relevance	Matches user query?
Context relevance	Uses relevant context?
Latency	Time to generate response

Common Challenges

- Retrieval may miss relevant info.
- Noisy or irrelevant context.
- Reranking adds latency.
- Hallucinations can still occur.
- Need good data and evaluation.

Why RAG?

- ✓ Up-to-date knowledge
- ✓ Less hallucination
- ✓ Better accuracy

Quick Revision

- ✓ RAG = Retrieval + Generation
- ✓ Use embeddings + vector DB
- ✓ Chunk smartly (with overlap)
- ✓ Rerank for better results
- ✓ Inject context in prompt
- ✓ Evaluate with multiple metrics

SUBTOPIC: LLM Agents & Tool Use

1. AI Agents

- AI agents are LLMs that can take actions, use tools and interact with environment to achieve goals.
- They go beyond text generation by planning, reasoning and executing tasks.
- Can work independently or with humans.

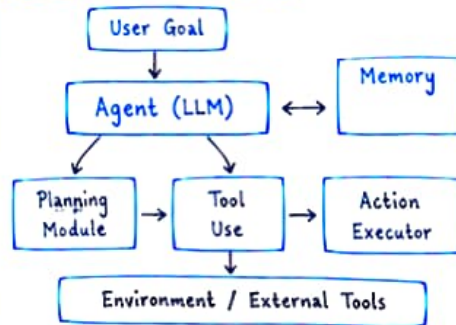
Key features:

- Goal-driven
- Tool use
- Multi-step reasoning
- Environment interaction

Example:

"Find flights to Delhi and book the cheapest one." → Agent uses search + booking tools.

2. Agent Architecture



Key components:

- LLM (reasoning + decision making)
- Tools (APIs, search, code, etc.)
- Memory (short + long term)
- Planner (breaks down tasks)

3. Tool Calling

- Lets the LLM use external tools / APIs to get real-time information or perform actions.
- Model decides when and which tool to use based on the user query.
- Returns tool output back to the model for further reasoning.

Example tools:

- Web search
- Calculator
- Code interpreter
- Database / API
- File system

Example:

User: "What's the weather in Pune?"
→ Agent calls weather API → returns result.

4. Function Calling

- A specific form of tool calling where the pre-defined function (function with structured inputs).
- Function executes and returns output in a structured format (e.g., JSON).
- Useful for integrating with programs and APIs.

Example:

function get_weather(city: string)
→ { temperature: float, condition: string }

5. Planning

- Breaks down a complex goal into smaller steps or subgoals.
- Can use ReAct, CoT or structured planning methods.
- Helps the agent decide the right order of actions and tools.

Types:

- Zero-shot planning
- Chain-of-thought planning
- Tree-of-thought planning

Example:

1. Search flights
2. Compare prices
3. Book ticket
4. Send confirmation

6. Memory

- Stores past information for better decisions and context.
- Can be short-term (conversation history) or long-term (user preferences, facts).
- Implemented using vector DBs, databases or key-value stores.

Types:

- Short-term memory
- Long-term memory
- Entity memory

Example:

Remember user's preference for vegetarian food across sessions.

7. Reasoning Loops

- Agent observes → reasons → acts → checks result → repeats if needed.
- Helps in complex, multi-step problems where the first attempt may not be correct.



Example: Debugging code, solving math problems, handling real-world tasks.

8. Multi-Agent Systems

- Multiple agents work together, share info and collaborate to achieve a common goal.
- Can be hierarchical, decentralized or peer-to-peer.
- Improves reliability, scalability and handles complex tasks.

Types:

- Supervisor-worker
- Peer-to-peer
- Hierarchical

Example:

One agent for research, one for coding, one for review.

9. Agent Frameworks

- Provide tools, libraries and structure to build and deploy agents.
- Handle tool integration, memory, planning and evaluation.
- Speed up development and reduce boilerplate code.

Popular frameworks:

- LangChain
- AutoGen
- CrewAI
- LlamaIndex

Example:

LangChain helps connect LLMs with tools, memory and agents.

10. Autonomous Workflows

- Agents can run end-to-end workflows without constant human intervention.
- Uses planning, tool use, memory and self-checking to complete tasks.
- Useful for automation, research, content creation, data analysis, etc.

Example: Research → Analyze → Write → Publish (all done by the agent)

Quick Comparison

Feature	Single Agent	Multi-Agent
Setup	Simple	Complex
Scalability	Limited	High
Task handling	Single task	Parallel / complex
Reliability	Lower	Higher
Use case	Basic automation	Advanced workflows

★ Key Takeaways

- Agents + tools = real-world actions.
- Planning + memory = better decisions.
- Multi-agent systems = more powerful.
- Frameworks = faster development.
- Autonomous workflows = end-to-end task execution.

★ Example: Research → language models from chatbots into real problem solvers. ★

SUBTOPIC: LLM Evaluation, Safety & Alignment

1. LLM Evaluation

- Measures how well an LLM performs on specific tasks and aligns with goals.
- Includes automatic metrics + human eval.
- Helps compare models and track improvements.

Key aspects:

- Quality (accuracy, relevance)
- Safety (harmful content, bias)
- Robustness (jailbreak resistance)
- Helpfulness (usefulness, clarity)

Example:

Evaluate a chatbot using MMLU, MT-Bench, Safety Bench + human rating.

2. Accuracy and Benchmarks

- Accuracy = % correct responses (task-specific).
- Benchmarks provide standardized datasets for comparison.
- Common benchmarks: MMLU, GSM8K, HELM, BIG-bench, TruthfulQA.

Benchmark	Focus	Type
MMLU	General knowledge	Multiple choice
GSM8K	Math reasoning	Text/MCQ
TruthfulQA	Factuality	Open-ended
HELM	Overall performance	Various

Key point: Benchmarks = real world performance.

3. Hallucinations

- Model generates false or unsupported info.
- Happens due to overconfidence, limited data or poor reasoning.
- Can be factual (wrong facts) or non-factual (made up details).

Example:

Q: What is the capital of France?
A: Berlin. (hallucination)

Mitigation:

- Better training data
- Retrieval (RAG)
- Fact-checking / verification

4. Toxicity

- Generates harmful, offensive or unsafe content.
- Can be intentional or unintentional.
- Measured using Perspective API, Detoxify, HateCheck, etc.



Key metrics:

- Toxicity score (0-1)
- Categories: hate, violence, sexual, self-harm, etc.

Example:

"Kill yourself" → High toxicity score

5. Bias

- Unfair or skewed behavior towards certain groups or perspectives.
- Can be in data, model or evaluation.
- Types: gender, racial, cultural, political, etc.

Example:

Model shows higher salary for men than women (gender bias)

Mitigation:

- Diverse data
- Bias evaluation tools
- Fairness constraints / debiasing

6. Alignment

- Ensures the model's goals match human values, intentions and safety requirements.
- Helps avoid harmful, misleading or undesired behavior.
- Can be explicit (rules) or implicit (RLHF, DPO).



Techniques:

- RLHF, DPO, Constitutional AI
- System prompts + safety rules
- Guardrails

7. RLHF

- Reinforcement Learning from Human Feedback.
- Uses human preference data to fine-tune model.
- Steps:



Benefit: Improves helpfulness, safety and alignment.

Example:

ChatGPT uses RLHF for better responses

8. DPO

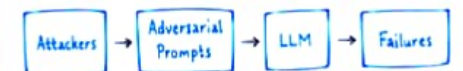
- Direct Preference Optimization.
- Aligns model using preference data without a reward model.
- Simpler and more stable than RLHF.

$$L_{DPO} = -E_{(x, y_w, y_l)} \left[\log \sigma(\beta(\log \pi_\theta(y_w|x) - \log \pi_\theta(y_l|x))) \right]$$

Key point: Directly Optimizes for preferences, NO reward model needed.

9. Red Teaming

- Tests model for vulnerabilities and harmful behavior.
- Uses adversarial prompts and creative attacks.
- Helps find Jailbreaks, bias, toxicity, etc.



Types: manual, automated, crowd-sourced.

Example:

Try bypassing safety with indirect prompts or roleplay.

10. Guardrails

- Safety layers around the model.
- Block harmful prompts or outputs
- Can be rule-based, ML-based or hybrid.

Types:

- Input filters
- Output filters
- Content moderation
- Policy enforcement

Example:

Block requests for illegal content or self-harm.

11. Responsible AI

- Builds trust and ensures ethical use of LLMs.
- Focuses on fairness, transparency, privacy, accountability and safety.
- Involves policy, evaluation and monitoring.

Key principles:

- Fairness
- Privacy
- Safety
- Transparency
- Accountability

Example:

OpenAI's usage policies, model cards, safety evals.

Quick Revision

- ✓ Evaluation = benchmarks + human eval.
- ✓ Hallucination = false info.
- ✓ Toxicity = harmful content.
- ✓ Bias = unfair behavior.
- ✓ Alignment = human values
- ✓ RLHF = human feedback.
- ✓ DPO = direct preference opt.
- ✓ Red teaming = find vulnerabilities.
- ✓ Guardrails = safety filters.
- ✓ Responsible AI = ethical + safe use.